Smallest enclosing balls by an active-set method on the dual

Thomas Schmelzer

September 3, 2026

Abstract

We record the dual quadratic program of the smallest enclosing ball problem, the active-set method that solves it, and the invariance properties an implementation must preserve. The method terminates at an exact vertex of the dual feasible set rather than at an interior-point tolerance, costs one $k \times k$ dense solve per iteration with $k \le d$, and returns the dual weights, which are the conic dual of the second-order cone formulation after one rescaling and hence a certificate the caller can check independently.

1 The problem and its dual

For $S = \{p_1, \dots, p_n\} \subset \mathbb{R}^d$ the smallest enclosing ball is the solution of

$$\min_{x,r} \; r \quad \text{s.t.} \quad \|p_i - x\| \le r, \; i = 1, \dots, n, \tag{1}$$

a second-order cone program in $(r, x) \in \mathbb{R}^{1+d}$ with n blocks $(r, p_i - x) \in \mathcal{Q}^{d+1}$. Existence follows from coercivity of $f(x) = \max_i \|p_i - x\|$ and uniqueness from strict convexity of f^2 along any segment; we write $(R, x^\star)$ for the optimum.

Let $\Delta_n = \{u \in \mathbb{R}^n : u \ge 0, \; \mathbf{1}^\mathsf{T} u = 1\}$, and for $u \in \Delta_n$ put $x(u) = \sum_i u_i p_i$ and

$$\varphi(u) = \sum_i u_i \|p_i\|^2 - \|x(u)\|^2, \tag{2}$$

a concave quadratic. Everything below rests on one identity, valid for all $y \in \mathbb{R}^d$ and $u \in \Delta_n$:

$$\sum_i u_i \|p_i - y\|^2 = \varphi(u) + \|x(u) - y\|^2. \tag{3}$$

Proposition 1. *$R^2 = \max_{u \in \Delta_n} \varphi(u)$, and $x^\star = x(w)$ for any maximiser w.*

Proof. If $B(y, r) \supseteq S$ then the left side of (3) is at most r^2, so $\varphi(u) \le r^2$ for every u; taking the optimum gives $\max \varphi \le R^2$. Conversely let $T = \{p_i : \|p_i - x^\star\| = R\}$. If $x^\star \notin \operatorname{conv} T$, separate: there is v with $v^\mathsf{T}(p - x^\star) > 0$ on T, and $\|p - (x^\star + tv)\|^2 = R^2 - 2tv^\mathsf{T}(p - x^\star) + t^2\|v\|^2 < R^2$ for small $t > 0$, while the finitely many points off T stay strictly inside — so the radius decreases, a contradiction. Hence $x^\star = \sum_i w_i p_i$ for some $w \in \Delta_n$ supported on T, and $y = x^\star$ in (3) gives $\varphi(w) = \sum_i w_i \|p_i - x^\star\|^2 = R^2$. The same substitution forces $\|x(w) - x^\star\|^2 \le 0$ for any maximiser. □

Since $\partial\varphi/\partial u_i = \|p_i - x(u)\|^2 - \|x(u)\|^2$, the first-order conditions for $\max_{\Delta_n} \varphi$ read, after cancelling the common term,

$$\|p_i - x\| = R \;\; (w_i > 0), \qquad \|p_i - x\| \le R \;\; (w_i = 0), \tag{4}$$

the geometric certificate: the points carrying weight lie on the boundary sphere and the rest are enclosed. By Carathéodory the support may be taken of size at most $d + 1$, so (1) is determined by at most $d + 1$ of the n points and the problem is combinatorial in the choice of that subset.

2 The method

Maintain a working set F and weights $u > 0$ on it summing to one.

Subproblem. Maximising φ over weights supported on F with no sign restriction is, by (4) without the inequalities, the requirement that x be equidistant from $\{q_j\}_{j \in F}$ and lie in their affine hull. With D the $(m-1) \times d$ matrix of edges $e_j = q_j - q_0$ and $x = q_0 + D^\mathsf{T} y$,

$$(DD^\mathsf{T})\, y = \tfrac{1}{2}\big(\|e_1\|^2, \dots, \|e_{m-1}\|^2\big)^\mathsf{T}, \qquad w = \Big(1 - \textstyle\sum_j y_j, \; y\Big), \tag{5}$$

a dense system of order $m - 1 \le d$, non-singular exactly when F is affinely independent. This is the circumcentre of the face, expressed in barycentric coordinates.

Dropping. If $\min_j w_j < 0$ the circumcentre lies outside $\operatorname{conv}\{q_j\}$ and the subproblem solution is dual-infeasible. Move along $s = w - u$ by the ratio test

$$\alpha = \min\{-u_i/s_i : \ s_i < 0\}, \tag{6}$$

which drives at least one weight to zero; remove it from F.

Degeneracy. If F is affinely dependent, DD^{T} is singular. The directions s with $\mathbf{1}^{\mathsf{T}}s = 0$ and $\sum_i s_i q_i = 0$ — equivalently $D^{\mathsf{T}}s_{1:} = 0$ after eliminating s_0 — leave $x(u)$ fixed, so $\|x(u)\|^2$ is frozen and $\varphi(u + ts) = \varphi(u) + t\sum_i s_i\|p_i\|^2$ is linear: the subproblem is unbounded. Project the gradient onto that null space and apply (6), shrinking F by one.

Adding. Otherwise accept $u \leftarrow w$, set $x = x(u)$ and $R = \max_{i\in F}\|p_i - x\|$, and let $k = \arg\max_i\|p_i - x\|$. If $\|p_k - x\| \le R$ then (4) holds and (R, x, u) is optimal; otherwise append k to F with weight zero. By (7) below this is also the steepest-ascent choice among the fixed variables.

> **Input** $p_1, \dots, p_n$. Recentre on $\bar{p}$. Set $F = \{k\}$, k farthest from p_1; $u_k = 1$.
> **Repeat:** if F affinely dependent, take the null-space descent step and drop; else solve (5). If $\min w < 0$, take the ratio step (6) and drop. Else set $u = w$, $x = x(u)$, $R = \max_{i\in F}\|p_i - x\|$; if no point exceeds R, return $(R,\ x + \bar{p},\ u)$; else drop zero weights and append $\arg\max_i\|p_i - x\|$ with weight 0.

Each iteration either strictly increases φ or strictly shrinks F, and there are finitely many working sets, so the method is finite; as with any active-set method a degenerate zero-length step requires an anti-cycling rule in exact arithmetic, and in practice an iteration cap serves as a safety net. Note that

$$\partial\varphi/\partial u_k - \partial\varphi/\partial u_i = \|p_k - x\|^2 - \|p_i - x\|^2, \tag{7}$$

so the farthest violator is the fixed variable with the largest reduced gradient.

3 Invariance

Two properties are load-bearing and easily lost. Both are consequences of refusing any tolerance tied to coordinates of magnitude one.

Origin invariance. Recentre on $\bar{p}$ before iterating, and form $\|p - x\|^2$ as $\sum_k (p_k - x_k)^2$ rather than $\|p\|^2 - 2p^{\mathsf{T}}x + \|x\|^2$. The expanded form cancels quantities of size $\|x\|^2$ to produce an answer of size R^2, losing every digit by which $\|x\|$ exceeds R; for a cloud of unit extent at distance 10^6 the rounding floor of a squared distance is then of the order of the radius itself.

Scale invariance. Test affine dependence on the rank of D, not on $[\mathbf{1}; Q^{\mathsf{T}}]$: the row of ones has unit entries, dominates the singular values, and would declare any cloud of extent below ε degenerate. Size the feasibility slack of the stopping test as a relative tolerance plus an absolute floor proportional to $\varepsilon \max_i\|p_i\|^2$, and normalise the null-space direction before comparing anything derived from it — the gradient carries units of length^2, the weight tolerance does not.

4 Cost, and the dual as a certificate

Per iteration the method performs one $O(nd)$ sweep of squared distances and one dense solve of order at most d; nothing of size n is ever factorised. This is the structural difference from an interior-point method applied to (1), which factors an $n(d+1)$-row system at every step. The method also identifies the support combinatorially and solves that face exactly, rather than approaching it from the interior and leaving the caller to decide which slacks of size 10^{-9} were meant to be zero.

The weights are the conic dual of (1) after one rescaling: the block for point i is

$$z_i = u_i\Big(1,\ -\frac{p_i - x}{R}\Big) \in \mathcal{Q}^{d+1}, \tag{8}$$

whose tail has norm $u_i\|p_i - x\|/R$, equal to its head exactly on the support, and which pairs with the slack $s_i = (R,\ p_i - x)$ as $s_i^{\mathsf{T}}z_i = (u_i/R)(R^2 - \|p_i - x\|^2) = 0$ — complementarity, each factor vanishing where the other does not. So a solver built on this method returns a standard-form primal–dual pair, and a caller can verify optimality from (4) at the cost of one pass over the data, without re-solving.

Remark (Substitutability). This is what makes the method usable behind a conic solver's interface rather than only as a special-purpose routine. The accompanying implementation dispatches on the cone list: zero and non-negative blocks go to a dense dual active-set QP method [4], and equally sized second-order blocks matching the shape of (1) go to the method above. A program carrying the right cones but a different matrix is refused rather than answered, which requires reconstructing the point cloud from (A, b) and rebuilding the program for comparison. An indefinite objective is likewise refused rather than regularised into definiteness, since $P + \epsilon I$ answers a different question.

5 A numerical comparison

Table 1 compares the method with Clarabel 0.11.1 applied to (1) and with Welzl's algorithm [8], on Gaussian clouds. Two quantities are reported, because either alone would mislead.

The first is the relative enclosure error $\max_i \|p_i - x\|/R - 1$, where a positive value means the returned ball does not in fact contain the cloud. It is a deterministic property of the returned pair, free of timing noise. Here the two basis-terminating methods are indistinguishable — both come in at zero or a single ulp, since each returns a centre from an exact subproblem solve and a radius that is by construction the maximum distance over its own support — while Clarabel reports its interior-point residuals, three to five orders larger. The exactness is thus a property of terminating at a basis, not of this method in particular; Section 7 is where the two basis methods actually differ.

The second is wall-clock time, and it must be read as a comparison of *implementations*. The method of Section 2 spends its time in one vectorised sweep per iteration, whereas Welzl's recursion evaluates a containment predicate per step in scalar Python; a good deal of the gap in the t columns is CPython against NumPy rather than one algorithm against another. The basis count is therefore given alongside: the number of circumspheres the recursion computes is a property of the algorithm and the input in any language.

		enclosure error			time (s)			
n	d	active set	Clarabel	Welzl	active set	Clarabel	Welzl	bases
10^3	2	0	$5.3 \cdot 10^{-12}$	$2.2 \cdot 10^{-16}$	0.000	0.008	0.010	265
10^3	5	$-2.2 \cdot 10^{-16}$	$-7.2 \cdot 10^{-12}$	$2.2 \cdot 10^{-16}$	0.000	0.033	0.018	592
10^4	3	0	$3.8 \cdot 10^{-12}$	0	0.001	0.174	0.445	2 406
10^4	20	0	$1.0 \cdot 10^{-12}$	—	0.006	0.934	—	—
10^5	2	0	$-8.0 \cdot 10^{-13}$	0	0.003	1.502	0.961	813
10^5	3	0	$1.3 \cdot 10^{-10}$	0	0.009	1.997	4.073	5 998
10^5	10	0	$-1.3 \cdot 10^{-12}$	—	0.015	6.972	—	—

Table 1: Standard normal clouds, one cloud per row; median of three runs. Apple M4 Pro, CPython 3.12.13, NumPy 2.5.1, Clarabel 0.11.1 at its default tolerances. Welzl was not attempted on the two rows marked —; Table 2 is where that limit is measured rather than guessed.

d	2	3	4	5	6	7	8	9	10	11
Welzl bases, median $/10^3$	0.19	1.05	2.50	6.4	12.0	21.4	58.6	115	189	276
Welzl bases, worst $/10^3$	0.55	1.70	4.92	12.9	35.9	49.3	194	577	864	1602
Welzl time (s)	0.006	0.027	0.053	0.143	0.234	0.490	1.174	2.593	3.714	5.493
active-set time (ms)	0.2	0.2	0.3	0.5	0.9	0.7	0.8	0.7	0.6	0.6

Table 2: Growth in d at $n = 10^3$ fixed: medians over 15 seeds, with the worst of those seeds on the second line. Medians are necessary here — the basis count is a high-variance random variable, and a mean over five seeds still puts $d = 11$ below $d = 10$.

Table 2 isolates the dependence on d at fixed n. Welzl's median basis count rises from 191 to 275 574 between $d = 2$ and $d = 11$, a factor between 1.5 and 2.7 per dimension across the upper half of that range, while the active-set time stays flat at 0.6–0.9 ms throughout: its work per iteration is a solve of order at most d, and d is small. The worst-seed column matters as much as the median. At $d = 11$ one of fifteen clouds cost 1.6 million basis computations, six times the median, so the distribution has a tail that a single measurement will not find — which is the practical form the expected-time analysis takes.

Neither comparison is entirely fair to its subject. Clarabel is built for large sparse programs of arbitrary shape and is here asked for a dense problem of one particular shape whose combinatorial structure the other

method exploits; Welzl's algorithm is being run in the least favourable language for a scalar recursion. That the specialised method wins on its own problem is the weakest possible claim, and the only one made here.

6 Relation to the minimum-norm point problem

If all points have equal norm ρ — for instance after the standard lift of a kernel problem onto a sphere — then (2) reduces to $\varphi(u) = \rho^2 - \|x(u)\|^2$, so maximising φ over Δ_n is minimising $\|x\|$ over $\operatorname{conv} S$: the minimum-norm point problem solved by Wolfe's algorithm and by the Frank–Wolfe family. The working set of Section 2 is then Wolfe's corral, the subproblem (5) is his affine minimiser, and the drop step is his major cycle. The general case is the same algorithm with the linear term $\sum_i u_i \|p_i\|^2$ restored, which is what makes the subproblem a circumcentre rather than a projection.

7 Relation to Welzl's algorithm

Welzl's randomised incremental method [8] computes the same object and, as Table 1 confirms, terminates at an exact basis just as the method above does. The two differ in how that basis is searched.

Welzl's search is combinatorial: it recurses over subsets, drawing a point p at random, solving without it, and re-solving with p forced onto the boundary when p falls outside the ball just computed. The permutation drives the recursion, the only arithmetic is the primitive that circumscribes at most $d + 1$ points, and the analysis is that of LP-type problems — expected $O(n)$ time for fixed d, with a constant known to grow superexponentially in d. The method of Section 2 searches instead by ascent on φ: the weights choose the pivot, a negative barycentric coordinate calling for a drop and the farthest violator for an addition. The search is monotone, local, deterministic, and admits add *and* drop steps; what it does not admit is an expected-time bound, and Section 2 claims none.

Table 2 is the practical shape of that difference: at fixed n the recursion's cost climbs by a factor of two or so per dimension and carries a long upper tail, while a solve of order d does not notice d in this range. The other difference is the certificate. Welzl's recursion yields the ball and its basis; the method above yields the barycentric weights as well, which are what make (4) checkable in one pass and what become (8) when a conic interface asks for a dual. The weights can be recovered from a basis by solving (5), but they are not a by-product of the search there, whereas here they *are* the search — an asymmetry that also makes a support set a natural warm start for a perturbed cloud, which a recursion over a fresh random permutation has no way to accept.

Gärtner's robust implementation [3] addresses the numerical fragility of the recursion's predicates and should be read alongside the original; the invariance requirements of Section 3 are this method's answer to the same question.

8 Notes

The problem is Sylvester's [6]; Elzinga and Hearn [1] gave an early finite algorithm of this type, and Section 7 places it against the randomised incremental line of work. The support-set framework of [2] also covers the smallest ball enclosing a set of *balls*; the method above is stated and implemented here for points only. The QP machinery is standard [5]. An implementation of the method, of the Goldfarb–Idnani core, and of the conic front end is available at `https://github.com/tschm/cvxball`.

References

[1] J. Elzinga and D. W. Hearn. Geometrical solutions for some minimax location problems. *Transportation Science* 6(4):379–394, 1972.

[2] K. Fischer, B. Gärtner and M. Kutz. Fast smallest-enclosing-ball computation in high dimensions. *Proc. ESA*, 630–641, 2003.

[3] B. Gärtner. Fast and robust smallest enclosing balls. *Proc. ESA*, 325–338, 1999.

[4] D. Goldfarb and A. Idnani. A numerically stable dual method for solving strictly convex quadratic programs. *Mathematical Programming* 27:1–33, 1983.

[5] J. Nocedal and S. J. Wright. *Numerical Optimization*, 2nd ed. Springer, 2006.

[6] J. J. Sylvester. A question in the geometry of situation. *Quarterly Journal of Pure and Applied Mathematics* 1:79, 1857.

[7] M. Frank and P. Wolfe. An algorithm for quadratic programming. *Naval Research Logistics Quarterly* 3:95–110, 1956.

[8] E. Welzl. Smallest enclosing disks (balls and ellipsoids). *New Results and New Trends in Computer Science*, LNCS 555, 359–370, 1991.

[9] P. Wolfe. Finding the nearest point in a polytope. *Mathematical Programming* 11:128–149, 1976.